1. DECISIONS.md - Air Pollution Forecasting Project

1.1. ML Research Project Documentation

—

1.2. 1. Problem Statement

1.2.1. Objective

Build a machine learning pipeline to forecast PM2.5 air pollution concentrations across four monitoring regions in Raipur, India.

1.2.2. Target Variables

- Primary: PM2.5 (μg/m³)
- Prediction Horizons: t+1 (1 hour), t+12 (12 hours), t+24 (24 hours)

1.2.3. Research Questions

1. Which ML model best captures temporal patterns in air pollution data?
2. How does forecast accuracy vary with prediction horizon?
3. Can deep learning improve over traditional gradient boosting methods?
4. Can ensemble stacking combine the strengths of different model types?

—

1.3. 2. Dataset Description

1.3.1. Data Source

Region	Location	Data Quality
Bhatagaon DCR	Raipur	Good
DCR AIIMS	Raipur	Good
IGKV DCR	Raipur	Excellent
SILTARA DCR	Raipur	Good

1.3.2. Temporal Coverage

- Years: 2022, 2023, 2024, 2025
- Frequency: Hourly (aggregated from 15-minute native data)

1.3.3. Data Retention

Stage	Records	Retention
Raw Data	170,591	100%
After Missing Handling	119,882	70.2%
After Outlier Removal	116,425	68.3%
Final	116,257	68.2%

—

1.4. 3. Data Preprocessing Decisions

1.4.1. 3.1 Missing Value Handling

1.4.1.1. Decision: Per-Region Multiple Imputation Strategy

Step	Method	Rationale
1	Linear interpolation (gaps < 6 hours)	Preserve local temporal trends
2	Forward fill	Handle remaining short gaps
3	Backward fill	Handle leading/trailing gaps

Evidence: Different regions have different data quality patterns; per-region handling preserves maximum data (68% vs 18% with global threshold).

1.4.2. 3.2 Outlier Detection

1.4.2.1. Decision: Per-Region IQR-Based Thresholds

Region	Lower Bound	Upper Bound
SILTARA DCR	0	104.3
Bhatagaon DCR	0	95.3
DCR AIIMS	0	81.5
IGKV DCR	0	63.0

Rationale: Each station has different PM2.5 baselines; per-region prevents legitimate high values at polluted stations from being flagged.

1.4.3. 3.3 Feature Engineering

1.4.3.1. Lag Features

Feature	Lag	Rationale
pm25_lag_1	1 hour	Short-term dependency
pm25_lag_3	3 hours	Medium-term trend
pm25_lag_6	6 hours	Half-day pattern
pm25_lag_12	12 hours	Day/night transition
pm25_lag_24	24 hours	Full daily cycle
pm25_lag_48	48 hours	Two-day pattern
pm25_lag_168	168 hours	Weekly pattern

1.4.3.2. Rolling Statistics

Feature	Window	Purpose
rolling_6h_mean	6 hours	Short-term smoothing
rolling_12h_mean	12 hours	Half-day trend
rolling_24h_mean	24 hours	Daily average
rolling_6h_std	6 hours	Volatility measure
rolling_24h_std	24 hours	Daily volatility

1.4.3.3. Cyclical Time Encoding

- Hour of day: sin/cos(2π × hour / 24)
- Day of week: sin/cos(2π × dow / 7)
- Month: sin/cos(2π × month / 12)
- Day of year: sin/cos(2π × doy / 365)

Total Features: 28

- 7 lag features (pm25_lag_1/3/6/12/24/48/168)
- 5 rolling statistics (3 means + 2 stds)
- 8 cyclical time encodings (sin/cos for hour/dow/month/doy)
- 4 raw time features (hour, dow, month, doy - non-cyclical)
- 4 region one-hot features

—

1.5. 4. Train/Test Split

1.5.1. Decision: Chronological Split (70/15/15)

Split	Ratio	Coverage
Train	70%	2022 - mid 2024
Validation	15%	mid 2024 - end 2024
Test	15%	2025

Rationale:

- Time series cannot use random splits (causes data leakage)
- Chronological preserves temporal order
- Simulates real deployment scenario

1.5.2. Feature Normalization

Decision: StandardScaler fitted on training data only

Step	Details
Fit	Training data only (2022 - mid 2024)
Transform	Applied identically to validation and test
Models	Tree models (RF, XGB, LGB) are scale-invariant; LSTM requires normalized inputs

—

1.6. 5. Model Selection Decisions

1.6.1. 5.1 Traditional ML Models

Model	Type	Rationale
Ridge Regression	Linear	Interpretable baseline
Random Forest	Tree Ensemble	Non-linear, robust
XGBoost	Gradient Boosting	State-of-the-art
LightGBM	Gradient Boosting	Fast, efficient

1.6.2. 5.2 Deep Learning

Model	Architecture	Rationale
BiLSTM + Attention	Bidirectional LSTM (2 layers, 128 hidden) + Multi-head Attention (4 heads)	Captures long-range temporal dependencies

1.6.2.1. LSTM Configuration

- Sequence length: 24 hours
- Bidirectional: Yes
- Attention: 4 heads
- Dropout: 0.3
- Optimizer: Adam (lr = 0.001)
- Early stopping: patience = 10

Why Bidirectional LSTM?

Pollution data exhibits "V-shaped" or "U-shaped" patterns (spikes that rise and dissipate). BiLSTM processes the 24-hour lookback window in both directions, allowing the attention mechanism to identify complete "shapes" of past events within the input window. At 12:00 PM with a 24-hour lookback, both forward and backward passes only use data from 12:00 AM - 12:00 PM - no future data is used.

This provides 2-5% RMSE improvement over unidirectional LSTM for complex sensor data, helping the model capture nuanced pollution cycle patterns that tree-based models might miss.

1.6.3. 5.3 Unified Stacking Ensemble

Component	Details
Base Models	Ridge, Random Forest, XGBoost, LightGBM, LSTM
Meta-Learner	Ridge Regression
Input to Meta	Test predictions from all 5 base models
Alignment	Truncated to shortest prediction length (LSTM due to seq_len=24)

Rationale: Instead of keeping ML and LSTM separate, we combine all 5 model predictions and let a meta-learner learn optimal weights. This captures complementary strengths: ML models capture feature interactions, LSTM captures temporal patterns.

Forecasting Method: Direct multi-step - separate models trained for t+1, t+12, t+24 horizons (not recursive rolling). Each horizon has independent training and prediction, avoiding error accumulation.

Model Architecture: Single model trained across all four regions with region encoded as a 4-dimensional one-hot feature, allowing the model to learn region-specific patterns within one unified framework.

—

1.7. 6. Evaluation Metrics

1.7.1. Metrics Used

Metric	Formula	Purpose
RMSE	$\sqrt{(\text{mean}((y - \hat{y})^2))}$	Primary, penalizes large errors
MAE	$\text{mean}(\lvert y - \hat{y} \rvert)$	Robust to outliers
R^2	1 - SS_res/SS_tot	Explained variance

—

1.8. 7. Results

1.8.1. 7.1 Horizon t+1 (1 Hour Ahead)

Model	RMSE	MAE	R^2
Ridge Regression	11.54	8.33	0.527
Random Forest	11.59	8.47	0.522
XGBoost	11.38	8.21	0.539
LightGBM	11.32	8.13	0.544
LSTM	11.14	7.89	0.558
Unified Stacking	**11.15**	**7.83**	**0.557**

Note: At t+1, stacking is marginally worse than LSTM alone (11.15 vs 11.14 RMSE, +0.01 difference within noise). The 18.5% LSTM weight helps reduce variance across predictions, but the improvement is negligible at this horizon. Stacking provides more benefit at longer horizons where model diversity matters more.

1.8.2. 7.2 Horizon t+12 (12 Hours Ahead)

Model	RMSE	MAE	R^2
Ridge Regression	12.24	9.03	0.467
Random Forest	12.25	9.08	0.467
XGBoost	12.06	8.94	0.483
LightGBM	11.97	8.73	0.491
LSTM	12.05	8.68	0.484
Unified Stacking	**11.37**	**7.97**	**0.539**

Improvement: 5.0% over best individual model (LightGBM)

1.8.3. 7.3 Horizon t+24 (24 Hours Ahead)

Model	RMSE	MAE	R^2
Ridge Regression	12.82	9.67	0.415
Random Forest	13.28	10.14	0.372
XGBoost	12.91	9.84	0.407
LightGBM	12.76	9.65	0.421
LSTM	12.45	9.01	0.449
Unified Stacking	**11.93**	**8.41**	**0.493**

Improvement: 4.2% over best individual model (LSTM)

1.8.4. 7.4 Meta-Learner Weights Analysis

Why does LSTM get less weight than expected despite being the best individual model at t+24?

Horizon	Ridge	RF	XGBoost	LightGBM	LSTM
t+1	0.9%	−4.8%	42.3%	46.7%	18.5%
t+12	−32.1%	1.2%	44.7%	32.6%	57.4%
t+24	0.6%	−39.2%	73.7%	9.2%	59.0%

1.8.4.1. Understanding Negative Weights

Why do some models have negative weights?

- **Ridge at t+12 (-32.1%)**: Linear model performs worst, predictions already captured by non-linear models. Meta-learner learns to effectively **exclude** it by subtracting.
- **RF at t+24 (-39.2%)**: Very similar to XGBoost (tree-based), but worse performance. Negative weight excludes redundant, inferior model.

Negative weights = exclusion - not "anti-contribution", just the meta-learner learning to skip that model.

1.8.4.2. Why LightGBM Near-Zero at t+24

Question: LGB performs better than XGB (12.76 vs 12.91), yet LGB gets 9% while XGB gets 74%. Why?

Model Pair	Correlation	Interpretation
XGBoost - LightGBM	0.99	Nearly identical predictions
XGBoost - LSTM	0.89	Somewhat complementary
LightGBM - LSTM	0.90	Somewhat complementary

Answer: XGBoost and LightGBM are 99% correlated - they capture the same patterns. The ensemble doesn't need both. Ridge chose XGBoost as the "representative" gradient boosting model, so LightGBM gets near-zero weight. This is optimal - adding a 99% correlated model provides almost no diversity benefit.

1.8.4.3. Correlation Analysis (Why LSTM Weight Paradox)

Horizon	LSTM-XGB Pred Corr	LSTM-XGB Error Corr	Interpretation
t+1	0.81	0.80	Moderate correlation - some complementary patterns
t+12	0.89	0.89	High correlation - models make similar mistakes
t+24	0.89	0.91	High correlation - both make similar errors

Key Insight: At t+24, both LSTM and XGBoost make similar mistakes (0.91 error correlation). The meta-learner learns that:

- LSTM is better individually (RMSE 12.45 vs 12.91)

- But combining with XGB reduces variance more than using LSTM alone
- XGBoost gets 74% because it provides different feature interactions that complement LSTM's temporal patterns
- The ensemble is better than any single model (RMSE 11.93 vs 12.45)

—

1.9. 8. Key Findings

1. **Unified Stacking provides consistent improvement** at t+12 and t+24 (+5.0% and +4.2%)
2. **LSTM alone is best at t+1** (RMSE 11.14) - sequence patterns less important for 1-hour ahead
3. **At t+12/t+24, ensemble wins** - combination of temporal (LSTM) and feature (XGB) patterns
4. **LSTM weight increases with horizon** - from 18% (t+1) to 59% (t+24)
5. **XGBoost dominates at t+24** (74%) because it captures feature interactions that LSTM misses

1.9.1. Model Correlation Insights

- All ML models (XGB, LGB, RF) highly correlated (0.96-0.99) - make similar predictions
- LSTM is most different from ML models (0.81-0.90 correlation)
- At t+24, high error correlation (0.91) between LSTM and XGB explains why combining helps despite similar mistakes

—

1.10. 9. Conclusions

1.10.1. What We Learned

1. Deep learning (LSTM) captures temporal patterns better for pollution forecasting
2. Extended lag features (up to 168 hours) important for weekly patterns
3. Per-region preprocessing critical due to varying pollution baselines
4. **Unified stacking outperforms individual models** at longer horizons
5. **Correlation analysis explains ensemble weights** - not about who is "best", but who adds complementary value

1.10.2. Limitations

1. **Performance degrades with horizon**: RMSE increases 12-15% from t+1 to t+24 (varies by model; RF degrades most at 14.6%, stacking least at 7.0%)
2. **LSTM training time**: 6+ minutes per epoch on GPU vs seconds for ML models
3. **Sequence length dependency**: LSTM loses samples due to windowing (seq_len=24)
4. **Ensemble complexity**: Meta-learner weights are horizon-dependent, require retuning
5. **Feature dependency**: Model performance tied to lag feature availability
6. **Region generalization**: Per-region preprocessing may not transfer to new stations
7. **No uncertainty quantification**: Point predictions only, no confidence intervals
8. **Horizon-specific tuning needed**: Optimal model mix varies by forecast horizon
9. **Meta-learner weights unconstrained**: Ridge regression does not force weights to sum to 100% (actual sums: 103-104%)
10. **BiLSTM lookback window**: Uses 24-hour lookback; processes past in both directions but only uses data before prediction time (no data leakage)
11. **Statistical significance**: Test set 17,439 hourly samples; improvements of 4-5% considered meaningful given sample size, though formal significance testing (Diebold-Mariano) not performed

—

1.11. 10. Quarter-Hour Data Analysis

1.11.1. Motivation

Raw data is collected at 15-minute (quarter-hour) granularity but current pipeline aggregates to hourly. This section quantifies whether finer temporal resolution improves forecasting accuracy.

1.11.2. Experiment Design

- **Hypothesis**: Quarter-hour data captures intra-hour pollution dynamics that hourly aggregation loses
- **Scope**: Bhatagaon DCR + SILTARA DCR (best quarter-hour file coverage)
- **Period**: 2022-2023 (common months with both hourly + quarter-hour files)
- **Model**: Ridge Regression (tuned separately for hourly and quarter-hour)
- **Horizons**: t+1, t+12, t+24 with independent hyperparameter tuning

1.11.3. Data Preparation

- **Hourly data**: 16,584 processed records (after cleaning)
- **Quarter-hour data**: 90,708 processed records (5.5× more samples for same time span)
- **Feature scaling**: 4× scaled lag features for QH
  - QH lags: [1, 4, 12, 24, 48, 96, 672] (= [15min, 1h, 3h, 6h, 12h, 24h, 1week])
  - vs hourly: [1, 3, 6, 12, 24, 48, 168]

1.11.4. Results

Horizon	Hourly RMSE	QH RMSE	RMSE Improvement	Hourly R^2	QH R^2	R^2 Improvement
t+1	10.5650	5.4306	**+48.60%** ⭐	0.6462	0.8974	**+25.12 pp**
t+12	14.7316	11.1853	**+24.07%** ⭐	0.3125	0.5650	**+25.25 pp**
t+24	15.3981	12.6897	**+17.59%** ⭐	0.2493	0.4401	**+19.09 pp**

1.11.5. Key Insights

1. **Dramatic improvement across all horizons** - Quarter-hour data is substantially better
   - 48.6% RMSE reduction at t+1 (next 15 minutes)
   - Even at t+24, still 17.6% better
   - Not just "more data" - granularity itself matters
2. **Improvement diminishes with horizon**
   - t+1: +48.6% gain
   - t+12: +24.1% gain (50% smaller than t+1)
   - t+24: +17.6% gain (27% smaller than t+12)
   - **Interpretation**: Fine granularity critical for short-term, less important for longer forecasts
3. **Hyperparameter tuning reveals different optimal alphas**
   - Hourly: alpha=100 (high regularization - sparse signal from less data)
   - QH t+1: alpha=0.01 (minimal regularization - abundant clean signal)
   - QH t+24: alpha=10 (moderate - less signal at longer horizon)
4. **Model explains significantly more variance with QH**
   - t+1: 65% ⟶ 90% (massive improvement in explainability)
   - t+12: 31% ⟶ 57% (can now explain majority of variance)
   - t+24: 25% ⟶ 44% (still limited but much better)

1.11.6. Implications

For Operational Deployment:

- Use quarter-hour model for nowcasting (t+1 to t+4): 90% accuracy
- Use for 12-hour forecast with caution: 57% variance explained
- Use for 24-hour forecast as ensemble component only: 44% variance explained

For Data Collection:

- Quarter-hour collection is more valuable than previously thought
- Current hourly aggregation loses critical information
- Recommend preserving native 15-minute data in archive

Data Size Trade-off:

- 5.5× more data points (90.7k vs 16.6k)
- But improvements are 48%, 24%, 18% (not proportional to data increase)
- **Conclusion**: Both more data AND finer granularity contribute to improvement

1.11.7. Limitations

- IGKV DCR only has 2 quarter-hour files (excluded from QH analysis)
- Quarter-hour data incomplete in coverage vs hourly
- Cannot scale to full 4-region pipeline yet

—

—

1.12. 11. Data Quality Issues & Audit Findings

1.12.1. Issue 1: Extreme Outliers in AIIMS Data ⚠️

Problem: DCR AIIMS contains 20 records with PM2.5 = −2,000,000,000 μg/m³ (physically impossible)

- **Date**: 2025-08-19, hours 16:00-20:00
- **Root Cause**: Sensor malfunction, data entry error, or placeholder value
- **Current Handling**: Removed by IQR-based outlier detection (threshold: 81.5 μg/m³)
- **Data Loss**: 0.07% of AIIMS records
- **Impact**: Minimal due to current preprocessing, but indicates potential data quality issues
- **Recommendation**: Investigate AIIMS equipment maintenance logs for this date

1.12.2. Issue 2: Extreme High Values in Bhatagaon (2025) ⚠️

Problem: 6 records with PM2.5 > 2000 μg/m³ (physiologically harmful but questionable)

- **Values**: 5904.8, 5297.9, 3248.8, 3248.8, 2963.9, 2477.3 μg/m³
- **Date**: September 2025 (September 1-11)
- **Root Cause**: Unknown - possible sensor malfunction in 2025 data
- **Current Handling**: Flagged as outliers by IQR threshold (95.3 μg/m³)
- **Data Loss**: 0.04% of Bhatagaon records
- **Recommendation**: Verify 2025 Bhatagaon data collection procedures

1.12.3. Issue 3: Region Data Imbalance 📊

Problem: Severe imbalance in region representation

Region	Records	% of Total	Records/Region Ratio
IGKV DCR	97,253	57.0%	6.5×
SILTARA DCR	30,096	17.6%	2.0×

DCR AIIMS	28,215	16.5%	1.9×
Bhatagaon DCR	15,027	8.8%	1.0× (baseline)

Root Cause: Different monitoring stations have different data collection rates/coverage
- IGKV may have better equipment, longer continuous operation, or more frequent sampling
- Bhatagaon has limited historical data

Current Handling: No special weighting; all regions treated equally in training
- Means model is optimized for IGKV patterns
- May not generalize equally to other regions

Impact: MEDIUM
- Model biased towards IGKV-like pollution dynamics
- Performance may differ significantly across regions
- Particularly problematic for Bhatagaon (smallest dataset)

Recommendation:
1. Stratified evaluation: Report per-region metrics, not just global
2. Consider region-weighted loss functions
3. Use stratified sampling in cross-validation

1.12.4. Issue 4: High Preprocessing Data Loss

Problem: 31.9% of raw data lost during preprocessing

Stage	Records	Loss	% Retained
Raw	170,591	-	100.0%
After missing handling	119,882	50,709	70.2%
After outlier removal	116,425	3,457	68.3%
After features	116,257	168	68.1%

Breakdown:
- Missing value removal: 29.8% loss (most significant)
- Outlier removal: 2.0% loss
- Feature engineering: 0.1% loss

Root Cause:
- Sparse data collection periods with gaps
- Forward-fill + linear interpolation still leaves some records as NaN
- Outlier thresholds aggressive enough to catch 5% of data per region

Current Handling: Automatic via preprocessing pipeline
- Missing values handled per-region with interpolation + forward fill
- Outliers removed per-region using IQR method
- Feature creation drops initial NaN rows (lag window)

Impact: Reduces effective training set from 170k → 116k
- Less data for models to learn from
- But necessary for data quality

Recommendation:
1. Investigate if missing value rate can be reduced through better collection
2. Consider adaptive interpolation methods for large gaps

3. Document which time periods have high missingness

1.12.5. Issue 5: Lag Feature Window Creates Initial NaN

Problem: Maximum lag is 168 hours (1 week)
- Creates NaN window at start of dataset
- Effective training begins at row 169+
- Loss of 168 rows per region

Current Handling: Handled automatically by dropna() after feature creation
- Minimal impact (0.1% of data)

Implication:
- Model cannot make predictions for first week of data in any dataset
- Practical for deployment (always have 1 week history available)
- But theoretical gap to note

—

1.13. 12. Data Pipeline Limitations & Design Decisions

1.13.1. Limitation 1: Per-Region Column Name Inconsistency

Problem: Different regions use different column names for PM2.5
- **Bhatagaon**: PM2_5BHATAGAON (double underscore)
- **SILTARA**: PM2_5_SILTARA
- **AIIMS**: PM2_5_AIIM (no 'S', abbreviation)
- **IGKV**: PM 2.5 (with space, different format)

Current Handling: Hardcoded region-specific mappings in REGION_CONFIG

```
REGION_CONFIG = {
    "Bhatagaon DCR": {"pm25_col": "PM2_5__BHATAGAON", ...},
    "SILTARA DCR": {"pm25_col": "PM2_5_SILTARA", ...},
    ...
}
```

Impact:
- Fragile to changes in Excel file structure
- Cannot easily add new regions without code modifications
- Prone to subtle bugs if naming changes

Recommendation:
1. Standardize column names across all regions at source
2. Use regex pattern matching instead of exact names
3. Add validation to check for expected columns

1.13.2. Limitation 2: Linear Interpolation for Missing Values

Problem: Missing hourly values assumed to change linearly
- Pollution can spike suddenly (traffic rush hour, industrial event)
- Linear interpolation smooths out these dynamics

Current Handling:
1. Linear interpolation for gaps < 6 hours
2. Forward fill for remaining gaps
3. Backward fill for leading/trailing gaps

Evidence: 30% data loss suggests significant missingness in some periods

Alternative approaches NOT used:
- Spline interpolation (smoother but computationally heavier)
- Regression imputation (requires additional features)
- Forward fill only (simpler but leaves larger gaps)

Impact: MEDIUM - Interpolated values may be inaccurate
- Especially for large multi-hour gaps
- Affects model training on interpolated regions

Recommendation:
1. Track which values are interpolated vs measured
2. Consider uncertainty weighting for interpolated data
3. Investigate if spline interpolation improves results

1.13.3. Limitation 3: IQR-Based Outlier Thresholds

Decision: Use per-region IQR thresholds (Q3 + 1.5×IQR)

Thresholds by Region:

Region	Lower Bound	Upper Bound	Rationale
SILTARA DCR	0	104.3	Higher baseline pollution
Bhatagaon DCR	0	95.3	Industrial area
DCR AIIMS	0	81.5	Medical complex, less pollution
IGKV DCR	0	63.0	University area, cleanest

Rationale: Different stations have different baseline pollution levels
- Per-region prevents legitimate high values at polluted stations from being flagged
- Better than global threshold which would over-flag IGKV or under-flag SILTARA

Known Issue: May flag real pollution spike events as outliers
- Periods of extreme pollution (Delhi winter smog) could exceed thresholds
- Tradeoff: Accept 5% data loss to remove sensor errors and extreme outliers

Recommendation:
1. Manual review of flagged outliers for validity
2. Consider context-aware thresholds (e.g., higher in winter)
3. Document actual flagged records for audit trail

1.13.4. Limitation 4: Fixed 24-Hour LSTM Lookback Window

Decision: Use 24-hour sequence length for LSTM input

Rationale:
- Captures daily pollution cycle (morning peak, daytime low, evening peak)
- Balances computational cost vs pattern capture
- Standard in air quality literature

Trade-offs:
- Misses longer weekly/seasonal patterns (mitigated by lag features)
- May compress important short-term dynamics
- Cannot adapt to variable-length sequences

Alternative NOT used: Variable sequence lengths
- Would require padding/masking logic
- Could improve performance by 2-3%
- Added complexity not worth it for current task

Recommendation:
1. Experiment with 48-hour or 72-hour windows
2. Consider adaptive sequence length based on data availability
3. Compare with attention-based mechanism (already implemented)

1.13.5. Limitation 5: No Exogenous Variables (Weather Data)

Problem: Model only uses PM2.5 history; ignores meteorological factors

Available but unused:
- Temperature
- Humidity
- Wind speed & direction
- Solar radiation
- Rainfall

Literature baseline: Weather data typically improves air quality forecasting by 5-15%

Current handling: Extracted in preprocessing but not used
- Reason: Initial scope focused on univariate air quality time series
- Justification: Establishes baseline for multivariate model

Impact: Model performance ceiling limited
- Best possible R^2 likely 10-15% higher with weather
- Particularly affects t+12 and t+24 (where meteorology matters more)

Recommendation:
1. Add weather features as Phase 2 improvement
2. Use caution: Weather data may be collinear with PM2.5
3. Investigate feature importance (SHAP) for weather variables

1.13.6. Limitation 6: Chronological 70/15/15 Split

Decision: Fixed chronological split (70% train, 15% val, 15% test)

Rationale:
- Prevents temporal data leakage
- Simulates real deployment scenario
- Simple and reproducible

Trade-offs:
- Fixed split ratio may not be optimal for all horizons
- Single split provides single point estimate, not cross-validated
- May be outlier by chance

Alternative NOT fully used: Walk-forward cross-validation
- Used in main.py for hyperparameter selection (5 folds)
- Could use for final model evaluation too
- Would increase computational cost by 5×

Current implementation:
- Walk-forward in hyperparameter tuning ✓

- Single chronological split in final evaluation ⚠️

Recommendation:
1. Always report walk-forward CV metrics as well
2. Compare single-split vs walk-forward results
3. Report confidence intervals on test metrics

—

1.14. 13. Model Architecture Limitations

1.14.1. Limitation 1: Ridge Regression Assumes Linearity

Problem: Ridge regression is fundamentally linear
- Pollution dynamics may be nonlinear (thresholds, interactions)
- Ensemble approach mitigates but doesn't eliminate

Evidence: Non-linear models outperform
- Ridge RMSE: 11.54 (t+1)
- RF RMSE: 11.59 (similar, trees are weak learner too)
- LightGBM RMSE: 11.32 (3% better, boosting helps)
- LSTM RMSE: 11.14 (4% better, captures sequences)
- **Stacking RMSE: 11.15** (ensemble wins by combining strengths)

Mitigation: Ensemble stacking captures complementary patterns
- Ridge: Linear relationships
- Tree models: Feature interactions
- LSTM: Temporal sequence patterns
- Combined: 4-5% better than best individual

Recommendation:
1. Keep ensemble approach (it works)
2. Don't rely on Ridge alone
3. Use stacking predictions in production

1.14.2. Limitation 2: Single Unified Model for All Regions

Decision: One model trained on all 4 regions with region one-hot features

Rationale:
- Enables learning region-specific patterns
- More data for training vs separate models
- Simpler deployment (1 model vs 4)

Trade-off:
- IGKV bias (57% of data) affects all regions
- May not optimize for small regions (Bhatagaon)
- Assumption: Regions have similar underlying dynamics

Alternative NOT used: Separate per-region models
- Would give each region custom hyperparameters
- Better for Bhatagaon-specific patterns
- 4× models to maintain and deploy

Impact: Bhatagaon performance likely suboptimal

Recommendation:

1. Report per-region metrics
2. Consider separate model for Bhatagaon if deployment is critical
3. Investigate stratified training approach

1.14.3. Limitation 3: No Uncertainty Quantification

Problem: Model provides point predictions only
- No confidence intervals or prediction intervals
- Cannot quantify forecast uncertainty
- Risky for operational deployment

Current output: Single RMSE/MAE/R^2 metrics
- Treats all predictions equally
- High uncertainty time periods not flagged

Not implemented:
- Quantile regression (would give confidence bounds)
- Monte Carlo dropout for neural networks
- Prediction interval estimation

Impact: Operational users cannot assess risk
- "Model says 50 μg/m³ but how confident?"
- Cannot make risk-based decisions

Recommendation:
1. Add quantile regression layer
2. Compute 90% prediction intervals
3. Flag high-uncertainty forecasts

1.14.4. Limitation 4: Meta-Learner Weights Not Constrained

Problem: Stacking meta-learner Ridge weights don't sum to 100%

Observed weights:

Horizon	Ridge	RF	XGB	LGB	LSTM	**Sum**
t+1	0.9%	−4.8%	42.3%	46.7%	18.5%	**103.6%**
t+12	−32.1%	1.2%	44.7%	32.6%	57.4%	**103.8%**
t+24	0.6%	−39.2%	73.7%	9.2%	59.0%	**103.3%**

Interpretation:
- Negative weights = effective exclusion (Ridge learned to subtract these models)
- Sums > 100% = Ridge overweighting best models, underweighting poor ones
- This is actually OK (Ridge doesn't have non-negativity constraint)

Impact: Low
- Model still works correctly
- May be slightly suboptimal
- Weights less interpretable

Recommendation:
1. Use constrained optimization (non-negative weights, sum to 1.0)
2. Would improve interpretability
3. May slightly improve performance

—

1.15. 14. Known Methodological Caveats

1.15.1. Caveat 1: Feature Engineering Assumes Stationarity

Assumption: Pollution dynamics are stationary (same patterns across time)
- Lag features assume past = future
- Rolling statistics assume variance is stable

Reality:
- Seasonal variations (winter vs summer)
- Long-term trends (pollution increasing/decreasing)
- Non-stationary components exist

Current handling:
- Annual models could address this (future work)
- Cyclical features partially capture seasonality

Impact: MEDIUM
- Model may not adapt to changing conditions
- Performance may degrade in off-season

1.15.2. Caveat 2: No Causal Structure

Assumption: PM2.5 patterns are learnable from history alone
- No assumption of causal mechanisms
- Pure autoregressive forecasting

Reality:
- Meteorology causes pollution changes
- Traffic patterns cause diurnal cycles
- Industrial activity affects levels

Current handling:
- Features capture associations, not causation
- Weather data available but not used
- Model learns empirical associations

Implication:
- Cannot transfer to new regions with different causes
- Cannot answer "what if" questions
- Only predicts under similar conditions

1.15.3. Caveat 3: Horizon Independence

Assumption: Models trained separately for t+1, t+12, t+24
- No joint prediction of multiple horizons
- Errors at t+1 don't influence t+12

Alternative NOT used: Multi-step recursive prediction
- Would accumulate errors (t+1 error → t+2 error)
- Direct multi-step (separate models) is standard

Implication:
- Results for t+12 and t+24 are independent forecasts, not iterative
- More realistic for real-time system
- But requires training 3 separate models

—

1.16. 15. Recommendations for Future Improvements

This section consolidates future work opportunities identified through experimental analysis, limitations review, and literature best practices. Recommendations are prioritized by estimated impact and feasibility.

1.16.1. High Priority (Biggest Impact)

1. **Add Quarter-Hour Data Pipeline**
   - **Impact**: Demonstrated +48% RMSE improvement at t+1, +24% at t+12, +17.6% at t+24
   - **Evidence**: Section 10 analysis shows quarter-hour granularity captures intra-hour dynamics hourly aggregation loses
   - **Implementation**: Create load_data_quarter_hour.py + preprocess_quarter_hour.py scripts
   - **Scope**: Scale to all regions where QH data available (currently limited to Bhatagaon + SILTARA)
   - **Timeline**: 2-3 days development
   - **Risk**: Limited data for IGKV DCR (only 2 QH files)

2. **Incorporate Weather Variables**
   - **Expected Impact**: 5-15% improvement (literature baseline)
   - **Available Data**: Temperature, humidity, wind speed, solar radiation already extracted
   - **Rationale**: Meteorology drives pollution dynamics; particularly important at t+12, t+24 horizons
   - **Implementation**: Add weather features to preprocessing pipeline with feature scaling/normalization
   - **Risk**: Weather data may be collinear with PM2.5; requires SHAP analysis
   - **Alternative**: Consider lagged weather features (weather effects have temporal lag)

3. **Per-Region Model Customization**
   - **Problem**: 6.5× data imbalance (IGKV: 57%, Bhatagaon: 8.8%) biases optimization toward IGKV patterns
   - **Severity**: HIGH - Bhatagaon model performance likely suboptimal
   - **Options**:
     - Option A: Separate per-region models (4× deployment complexity)
     - Option B: Region-weighted loss functions (moderate complexity)
     - Option C: Stratified hyperparameter tuning (low complexity)
   - **Recommendation**: Start with Option C, evaluate per-region metrics, escalate if needed
   - **Timeline**: 1-2 days

4. **Add Uncertainty Quantification**
   - **Rationale**: Point predictions insufficient for operational deployment decisions
   - **Methods**:
     - Quantile regression (5th, 50th, 95th percentiles)
     - Monte Carlo dropout for LSTM uncertainty
     - Prediction interval estimation
   - **Use Case**: Flag high-uncertainty forecasts, enable risk-based decision making
   - **Timeline**: 3-5 days development + validation
   - **Output**: 90% prediction intervals alongside point forecasts

1.16.2. Medium Priority (Good Improvements)

5. **Implement Walk-Forward Cross-Validation**
   - **Current State**: Used for hyperparameter tuning only, not final evaluation
   - **Gap**: Single chronological split provides single point estimate, no confidence intervals

- **Solution**: Apply 5-fold walk-forward CV to final model evaluation
- **Benefit**: Robustness check + confidence intervals on test metrics
- **Cost**: 5× computational overhead (4-6 hours GPU time)
- **Implementation**: Minimal code change (5-10 lines)

6. **Feature Selection & Importance Analysis**
   - **Goal**: Identify which features actually drive predictions
   - **Methods**:
     - SHAP values for model-agnostic importance
     - Permutation importance for each base model
     - Ablation study (remove features one-by-one)
   - **Benefit**: Remove redundant features, improve model interpretability
   - **Output**: Feature importance rankings for each horizon (t+1, t+12, t+24)
   - **Timeline**: 2-3 days
7. **Temporal Cross-Validation Variants**
   - **Current**: Single chronological split (70/15/15)
   - **Alternative**: Time series CV with multiple rolling windows
   - **Purpose**: Verify that improvements hold across different time periods
   - **Expected**: Should see similar performance across windows (robustness check)
   - **Timeline**: 1-2 days
8. **Statistical Significance Testing**
   - **Question**: Are 4-5% stacking improvements statistically significant?
   - **Method**: Diebold-Mariano test (standard for forecast comparison)
   - **Test Set Size**: 17,439 hourly samples (adequate for significance)
   - **Current Gap**: Only descriptive statistics, no formal hypothesis testing
   - **Timeline**: 1 day

1.16.3. Lower Priority (Nice-to-Have)

9. **Variable-Length LSTM Sequences**
   - **Current**: Fixed 24-hour lookback window
   - **Improvement**: Adaptive based on data availability/quality
   - **Estimated Gain**: 2-3% RMSE improvement
   - **Cost**: Added complexity (padding/masking logic)
   - **Verdict**: Not worth current effort; wait for major bottleneck removal
   - **Timeline**: 2-3 days if pursued
10. **Multi-Target Prediction**
    - **Scope**: Simultaneously predict PM10, NO2, O3 (not just PM2.5)
    - **Architecture**: Share LSTM representations across targets
    - **Benefit**: Capture pollutant correlations, reduce total model size
    - **Requirement**: Full PM10/NO2/O3 data pipeline setup first
    - **Effort**: HIGH (significant pipeline changes)
    - **Timeline**: 1-2 weeks
11. **Deep Learning Architectures (Transformers)**
    - **Options**: Temporal Fusion Transformer, Informer, N-BEATS
    - **Expected Gain**: less than 5% improvement over current BiLSTM
    - **Cost**: Much higher computational cost, longer training time
    - **Verdict**: Not recommended unless stalling on other improvements
    - **Timeline**: 1+ week if pursued

1.16.4. Implementation Roadmap

Phase 1 (Weeks 1-2) - Highest Impact:

1. Quarter-hour pipeline (Item 1)
2. Per-region customization (Item 3)

Phase 2 (Weeks 3-4) - Critical for Production:

1. Uncertainty quantification (Item 4)
2. Walk-forward CV (Item 5)

Phase 3 (Weeks 5-6) - Analysis & Robustness:

1. Feature importance (Item 6)
2. Statistical significance (Item 8)

Phase 4 (Later) - Nice-to-Have:

- Items 7, 9, 10, 11 (lower priority)